Assessment Cover Sheet

Programme Name:	FdSc / Cert HE Computing
Partner University Name:	NCG
Semester 1 or 2:	2
Module Code:	S2-PFD200
Name of Lecturer:	Muhammad Majid Ali
Due Date:	12.06.2026
Assessment Type:	Report & Practical (Task-1,2 &3 Development)

Declaration of Authenticity (please check and sign):

By submitting this assignment, I declare that this work is fully my own, and the work of others (including internet sources) is acknowledged by quotations and appropriate citing and referencing.

I declare that this work has not made use of the work of any other student(s) past or present at this or any other educational institution or source.

I declare that I have not commissioned another person to complete this work. This could include the use of professional essay-writing services, essay banks, or ghostwriting services.

I declare that I have not asked another person to complete this work for me. This could include the help of friends, classmates, other students, or family members.

I declare I have not used any AI software to help write this work outside of the assessment brief/guidelines (this includes translation tools, paraphrasing tools, or content creation websites such as 'ChatGPT'), other than for proofreading.

I acknowledge that I have read all the texts listed in the bibliography or references list. I confirm all sources cited in the work appear in the bibliography or references list at the end.

Signed: Jurijs Petkevics

Table of Contents

TASK 1: PORTFOLIO

Part 1: Self-Assessment

Before thinking about work in web development, software development or mobile application development, I needed to stop for a little bit and honestly look at myself from the side. Not like a person who already knows everything, but like a computing student who already understood something, made some mistakes, and in some places still moves carefully, like in a dark room trying to find the light switch. This is the meaning of my self-assessment. I want to understand what skills I already have, what I can do more calmly, and what still needs time and normal practice. I also explain why I chose Python for my mini-project. It was not a random choice. More likely, I finally accepted that this gap was standing in front of me for a long time, like a closed door, and now it is time to open it.

If I speak about what is closest to me, then it is a mix of visual design and development. Not just code for the sake of code. It is important for me how everything looks on the screen, how a person clicks a button, how his eyes move through the page, where colour helps, and where it only gets in the way. Layouts, fonts, colours, the general mood of the interface - all this was always a little bit easier for me to understand than dry programming logic. I worked with Photoshop, CorelDRAW and Capture One, and these programs are not strange for me. In Photoshop it is comfortable to collect visual ideas, CorelDRAW helps with graphics, and Capture One I use more often for images. When I need to prepare materials for an interface or just make a picture look better, these tools help a lot.

Design for me does not live somewhere separate from development. I do not see it as two different kingdoms, with a stone wall standing between them. Of course, an application must work correctly. If a button does not click, the database crashes, and the form does not save a product, then a nice colour will not save it anymore. But the other side is important too. If the program works, but looks heavy, unclear and like an old cupboard with crooked doors, a person will get tired quickly. A good interface should help, not argue with the user. I did not understand this straight away. At some moment it just clicked: code and appearance should go together, like two horses in one harness.

In web development, I already have some base. I do not call myself an expert, it would sound too loud. But HTML, CSS, PHP and MySQL I understand on a level where I can build a working system, not only a beautiful page with no life inside. It became more clear to me how a form sends data, what the server does with it, how PHP speaks with the database and why frontend without normal backend is often like a shop window with no shop behind it. I understood especially a lot during the work on FixerUpper. There was everything at once: products, stock, orders, checkout pages, tables, connections between data. Sometimes it felt like I hold a ball of threads in my hands and do not know which end to pull. But exactly there many things finally became alive. XAMPP, phpMyAdmin, SQL queries stopped being just words from lessons. They became tools which I really worked with.

Java stands a little bit on the side for me. I studied it, worked with classes, methods, encapsulation, and made simple desktop applications. But honestly - Java did not feel as natural as web development. With it I had more inner resistance. Sometimes I understood the idea, but the code itself looked too strict, like it always demands: “stand straight, speak correctly”. And still Java gave me an important lesson. It taught me to think about structure. Before, I could just start writing code and hope that later I will understand it. But then comes that moment when you need to fix something, and all the code turns into a swamp. With Java I understood better why classes, order, and separation of logic are needed. Even if Java will not become my main direction, this lesson stayed with me.

Python was the most visible weak place for me. I heard and read a lot about it. Automation, data analysis, artificial intelligence, cybersecurity, scripts, desktop tools - Python appears almost everywhere. I understood the theory more or less. But to understand a language in words and to make a normal thing with it - these are different roads. You can talk about a bicycle, but until you sit and ride, balance will not appear by itself. With Python, it was exactly like this for me. I knew that it is important for a future developer, especially if you want to be flexible and not get stuck in one narrow corner. So this gap could not just be avoided anymore.

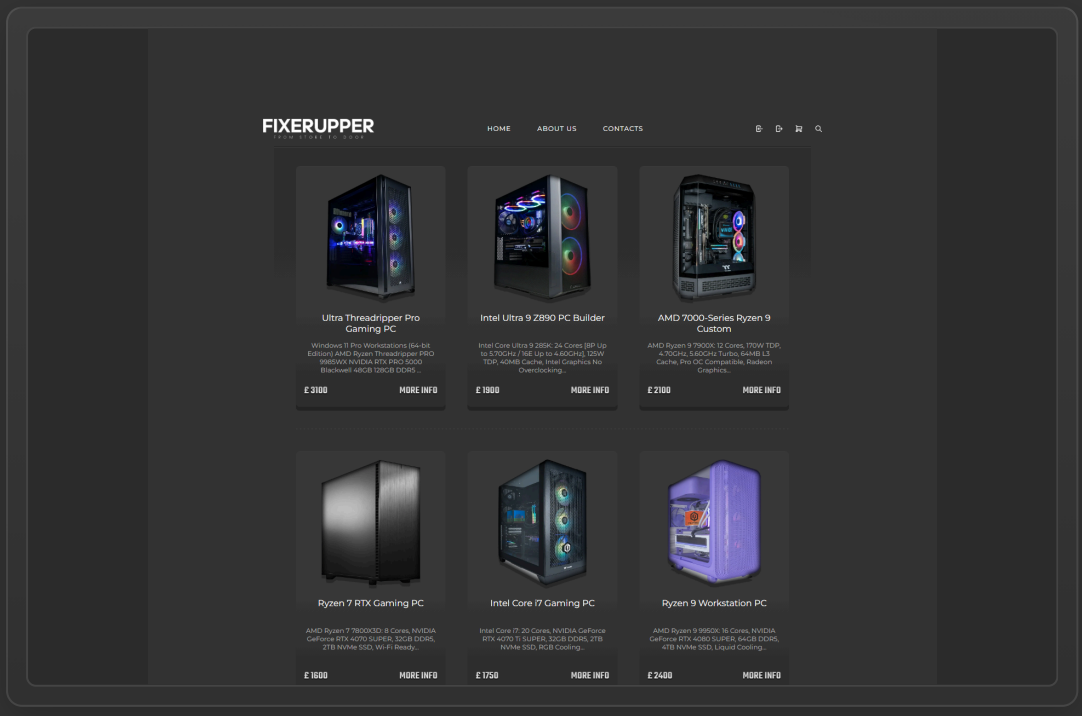
With documentation and planning, everything is also not fully smooth for me. I say this directly, without decorations. When a new idea appears, I want to open the editor straight away and start writing code. There is some excitement in this. But in real development, you will not go far like this. First you need to understand the requirements, split the task into parts, think about the database, about the user, about testing, and about how to explain your decisions later. Code is not all the work, even if it often looks like the main part. Documentation sometimes looks boring, like grey rain outside the window, but later it saves you. When you need to remember why you did it exactly this way and not another way, normal notes become almost like a treasure map.

If I look forward, I want to develop in web development, software development or mobile application development. The best role for me is where there is both technical part and visual side. I like not only to make a program work, but also to make it nice to use. PHP, MySQL, Java, Flutter and design already give me a base. Not perfect, but real. Now I needed to show that I can take a new area and not just watch few videos, but make something useful. That is why Python became the right choice for this project. A few small exercises would not prove almost anything. I needed to make a tool with a real purpose, which is connected with an existing system and works with a real database.

Part 2: Project Proposal and Negotiation

When I finished my self-assessment, the direction became quite clear. Not straight away loud and beautiful, but it became clear. Python was the place where I needed to improve. I was putting it away for too long, like a book on the shelf which you always promise to read. I could take a simple tutorial, repeat the exercises and say that I practised. But I wanted something closer to real work. Where there is a goal, mistakes, limits and the need to finish the thing until the end.

FixerUpper already existed as a project, and I understood its idea quite well. So the thought to make a desktop admin tool for it looked reasonable. It was not a project from empty place. It was connected to an already made system. It meant that I was learning Python not in vacuum, but inside a context which I understand. This gave more meaning straight away. I noticed that it is easier to learn like this. When a task is not hanging in the air, but really serves something, the brain holds on to it stronger.



This is how FixerUpper Inventory Desktop appeared. It is a desktop Python application on Tkinter. Its task is simple, without extra fancy things: to give an administrator a comfortable interface for managing products in the FixerUpper storefront. Create a product, edit it, duplicate a record, hide an item - all this can be done through GUI. It is not needed every time to go by hands into SQL and hope that you did not make mistake in one letter. For a person who does not want to live inside phpMyAdmin, this tool already looks much calmer.

Before this, products could be changed through phpMyAdmin or direct SQL queries. It works, but the way is quite harsh. It needs technical knowledge, time and attention. One small mistake can damage the data. GUI on top of the same MySQL database looked like a more human solution. The data stays the same, the tables stay the same, but between the person and the database there is a layer which checks input and guides the process. Like handrails on stairs: you can walk without them, but with them there is less chance to fall.

This project made me not just read about Python, but really use it. Functions, classes, dataclasses, exceptions, files, input validation - all this came out from study pages and became part of the work. Tkinter was also new land for me. Web interface and desktop interface feel different. In browser, many things are already done for you. In Tkinter you need to think by hands about buttons, windows, lists, events and position of elements. At first it confused me a little. Then, when the first elements started to behave how they needed, I had a feeling: yes, this is it, finally it clicked.

The hardest part became the database. A full product in FixerUpper is not one row. It is connected with several tables. It is needed to write main information, stock details, image path and specifications. If something breaks in the middle, you cannot leave the database in a strange state. Here I first felt much stronger why transaction logic and carefulness are needed. Before, a database looked just like a place where data is stored. In this project it became more like a mechanism with small gears. You turn one - another one moves.

MySQL database, check data before saving, make the interface in the FixerUpper style and describe the process properly. Not at the very end, when everything is already forgotten, but during the work. This was important because documentation is also part of professional development. And yes, sometimes writing an explanation was harder than the code itself. Code at least complains with an error. But text just looks at you like an empty page.

I separated the work into stages, although in real life everything did not go by perfectly straight road. First I chose Tkinter, because it comes together with Python and does not need extra installation. For local environment with XAMPP, it was comfortable. Then I thought about the tables: products, product_inventory, product_images and product_specs. After that came the interface, then validation logic, saving data, testing, reflection and documentation. On paper it looks very tidy. In life I had to go back, fix things, doubt, and check again.

After discussion with the teacher, the project was confirmed as a Python desktop administration tool, not just a small programming exercise. For me this was important. The project got a normal size: not too small to be empty, and not too huge to drown in it.

TASK 2: PROJECT DEVELOPMENT

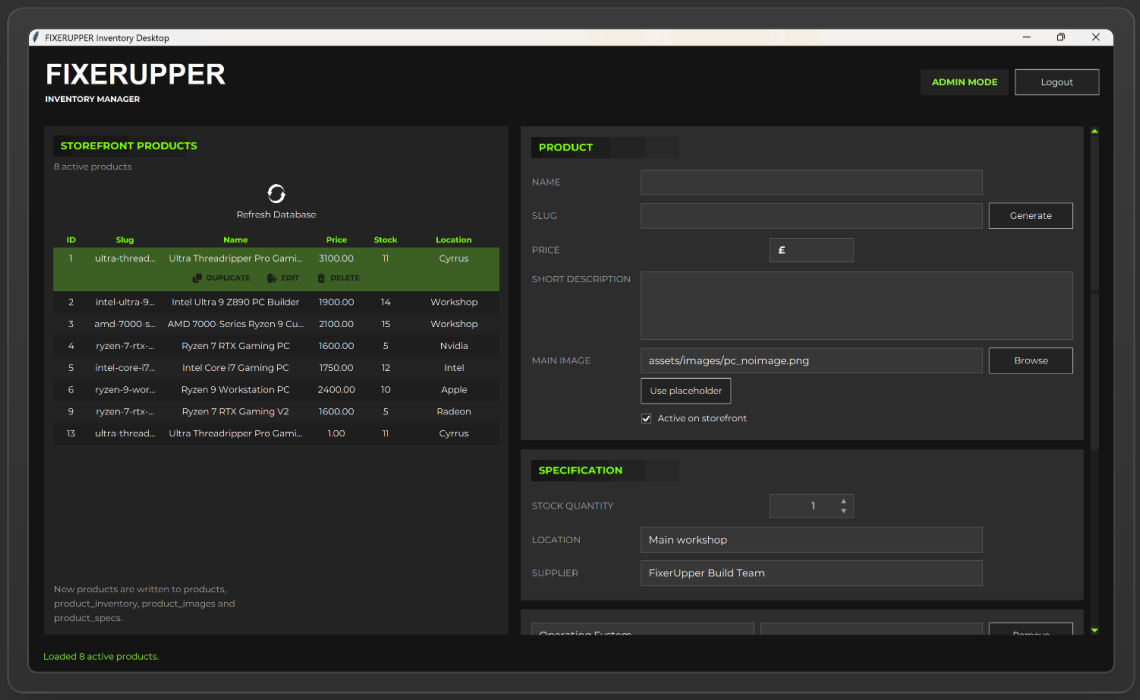
Project Overview

The finished application became a desktop Python tool for managing products in the FixerUpper storefront. Tkinter is responsible for the graphical interface. The application works locally through XAMPP and speaks with MySQL database not through a separate Python package, but through mysql.exe from XAMPP, using subprocess. This decision is not the most beautiful one for a big production project, but for my local prototype it worked and helped me understand the process deeper.

The application solved a real problem. The FixerUpper website shows products to customers, but creating these products inside the database is not one simple step. A product has stock data, image link, specifications for modal window and other parts. If doing this by hand, it is easy to forget one table or write a wrong value. My desktop tool collects these actions into one controlled process. Not magic, of course, but order instead of chaos.

The application has admin login, live product list, create product form, editing, duplicate option, soft delete, image selection and copying, automatic slug generation, specification rows, full input validation, transaction-based saving and command line health check through --check-db. Separately, it sounds like a list of small things. Together, they make the tool look like a real admin panel, only desktop.

Design and User Interface



Visually, I wanted to keep the connection with the FixerUpper brand. Dark grey backgrounds, white text, lime green accents - all this was already part of the general style. I did not want the admin tool to look like a strange relative who was accidentally put at the same table. It had to be from the same family as the storefront.

I divided the screen into two main parts. On the left there is a product list from the database: ID, slug, name, price, stock and location. This way the administrator can immediately see what is already in the system. On the right there is a scrollable form. There you can enter product name, slug, price, description, image, active status, stock amount, location, supplier and modal specs. Sections Product, Specification and Modal Specs help not to drown in the fields. Although, to be honest, when the form became long, I first got a little scared myself. But then it became clear: the product itself is complex, and the interface just shows this complexity in a more tidy way.

Normal Tkinter widgets look quite simple. Sometimes even too simple, like somebody forgot to dress them before going outside. Because of this, for some buttons I used Canvas components and made custom buttons with icons. This is a small detail, but it changes the feeling from the program. The tool starts to look less like a study exercise and more like a thing which can really be used.

Technical Implementation

In the project I used standard Python modules: argparse, csv, hashlib, hmac, os, re, shutil, subprocess, sys, dataclasses, Decimal, Path and Tkinter. Each of them had its own job. Path helped with paths, shutil copied images, subprocess started mysql.exe, Decimal was needed for prices, and Tkinter kept the whole interface. I liked that many things could be done with the standard library. It is not needed to pull a mountain of external packages straight away.

Standard library modules and project constants — tools/fixerupper_inventory_desktop.py

```python
import argparse
import csv
import hashlib
import hmac
import os
import re
import shutil
import subprocess
import sys
from dataclasses import dataclass
from decimal import Decimal, InvalidOperation
from pathlib import Path
from typing import Iterable

import tkinter as tk
import tkinter.font as tkfont
from tkinter import filedialog, messagebox, ttk

PROJECT_ROOT = Path(__file__).resolve().parents[1]
ASSETS_IMAGES_DIR = PROJECT_ROOT / "assets" / "images"
REFRESH_ICON_PATH = ASSETS_IMAGES_DIR / "refresh_icon.png"
CLONE_ICON_PATH = ASSETS_IMAGES_DIR / "clone.png"
PENCIL_ICON_PATH = ASSETS_IMAGES_DIR / "pencil.png"
TRASH_ICON_PATH = ASSETS_IMAGES_DIR / "trash.png"
BASE_DPI = 96.0
PRODUCTS_PANEL_WIDTH_SCALE = 1.04
HEADER_GRADIENT_WIDTH = 200
HEADER_HEIGHT = 28
HEADER_TEXT_PAD_X = 10
REFRESH_ICON_SIZE = 50
ACTION_ICON_SIZE = 25
ICON_TEXT_GAP = 6
PRODUCT_ACTION_BUTTON_GAP = 14
PRODUCT_TABLE_COLUMNS = ("id", "slug", "name", "price", "stock", "location")
PRODUCT_NUMERIC_COLUMNS = {"id", "price", "stock"}

# Connection defaults match the local XAMPP setup used by config.php.
DEFAULT_MYSQL_EXE = Path(r"C:\xampp\mysql\bin\mysql.exe")
MYSQL_EXE = Path(os.environ.get("FIXERUPPER_MYSQL_EXE", str(DEFAULT_MYSQL_EXE)))
MYSQL_DATABASE = os.environ.get("FIXERUPPER_DB_NAME", "fixerupper")
MYSQL_USER = os.environ.get("FIXERUPPER_DB_USER", "root")
MYSQL_PASSWORD = os.environ.get("FIXERUPPER_DB_PASSWORD", "")
```

Code Screenshot 1. Standard Python modules, Path-based project folders, table constants and XAMPP/MySQL connection defaults used by the desktop tool.

I moved constants separately: paths, colours, column names, allowed image types and default values. This made the code a little longer, but more clear. Dataclasses also helped to put data on small shelves: ProductRow for the list, ProductPayload for checked data, ProductDetails for editing a product and AuthenticatedUser for the current session. At first it looked like extra strictness. Then it became clear that without these small boxes, data quickly starts walking anywhere it wants.

Dataclasses for structured product data tools/fixerupper_inventory_desktop.py

```python
@dataclass
class ProductRow:
    """Small read model used by the left-hand "Existing products" table."""

    product_id: str
    slug: str
    name: str
    price: str
    stock: str
    location: str


@dataclass
class ProductPayload:
    """Validated form data ready to be written to the FixerUpper schema."""

    slug: str
    name: str
    short_description: str
    price: Decimal
    main_image: str
    is_active: bool
    stock_quantity: int
    location: str
    supplier: str
    specs: list[tuple[str, str]]


@dataclass
class ProductDetails:
    """Full product data loaded from the database for editing."""

    product_id: int
    payload: ProductPayload


@dataclass
class SpecControls:
    """Tkinter widgets and variables for one editable specification row."""

    frame: ttk.Frame
    label_var: tk.StringVar
    value_var: tk.StringVar


@dataclass
class AuthenticatedUser:
    """Authenticated desktop user loaded from the users table."""

    username: str
    email: str
```

Code Screenshot 2. Dataclasses keep product rows, validated payloads, edit details, specification controls and authenticated user data in clear structures.

DatabaseClient became the place where all work with MySQL lives. It builds arguments for mysql.exe, sends SQL through stdin, reads output and throws DatabaseError if something went wrong. GUI already catches these errors and shows them in more understandable language. There are methods ensure_auth_schema, authenticate_admin, list_products, get_product, create_product, update_product and soft_delete_product. I separated database logic from the interface on purpose. And this decision saved me more than once later. When I needed to change one part, I did not have to break everything else.

DatabaseClient wrapper around mysql.exe tools/fixerupper_inventory_desktop.py

```python
class DatabaseClient:
    """Thin wrapper around XAMPP's mysql.exe command-line client.

    A normal desktop application would often use a DB-API package such as
    ``mysql-connector-python`` or ``PyMySQL``. This project avoids that dependency
    so the tool can be launched immediately after cloning the repository on the
    same Windows/XAMPP environment as the PHP site.
    """

    def __init__(
        self,
        mysql_exe: Path = MYSQL_EXE,
        database: str = MYSQL_DATABASE,
        user: str = MYSQL_USER,
        password: str = MYSQL_PASSWORD,
    ) -> None:
        self.mysql_exe = mysql_exe
        self.database = database
        self.user = user
        self.password = password

    def _args(self) -> list[str]:
        """Build the mysql.exe command arguments used for every request."""

        args = [
            str(self.mysql_exe),
            "--batch",
            "--raw",
            "--skip-column-names",
            "--default-character-set=utf8mb4",
            f"--user={self.user}",
        ]

        if self.password:
            args.append(f"--password={self.password}")

        args.append(f"--database={self.database}")
        return args

    def run(self, sql: str) -> str:
        """Execute SQL and return stdout, raising DatabaseError on failure.

        SQL is passed through ``stdin`` rather than shell interpolation. This
        keeps command execution predictable on Windows and avoids quoting issues
        with paths, product names, or long specification text.
        """

        creationflags = subprocess.CREATE_NO_WINDOW if hasattr(subprocess, "CREATE_NO_WINDOW") else 0
        process = subprocess.run(
            self._args(),
            input=sql,
            text=True,
            encoding="utf-8",
            errors="replace",
            capture_output=True,
            cwd=PROJECT_ROOT,
            creationflags=creationflags,
            check=False,
        )

        if process.returncode != 0:
            details = process.stderr.strip() or process.stdout.strip() or "unknown mysql error"
            raise DatabaseError(details)

        return process.stdout
```

Code Screenshot 3. DatabaseClient builds mysql.exe arguments, sends SQL through stdin with subprocess.run, captures output and raises DatabaseError on failure.

I used transactions for create and update operations. One product touches four tables, and this is a dangerous place. If one insert worked, but another one did not, the database can become crooked. Transaction makes it so that the operation either finishes fully, or rolls back. For me it was almost like a rule of fair deal: either everything together, or nothing.

Transaction-based product creation tools/fixerupper_inventory_desktop.py

```
    def create_product(self, payload: ProductPayload) -> int:
        """Insert a new product and all related records in one transaction.

        The PHP storefront expects several linked tables to be present for a rich
        product card: ``products`` for the core listing, ``product_inventory`` for
        saleable stock, ``product_images`` for the modal gallery, and
        ``product_specs`` for the text details. Creating all of them together
        prevents a half-configured product from appearing on the site.
        """

        spec_sql = []

        for sort_order, (label, value) in enumerate(payload.specs, start=1):
            spec_sql.append(
                "INSERT INTO product_specs (product_id, label, value, sort_order) "
                f"VALUES (@fixerupper_product_id, {sql_quote(label)}, {sql_quote(value)}, {sort_order});"
            )

        sql = f"""
        SET NAMES utf8mb4;
        START TRANSACTION;

        INSERT INTO products (slug, name, short_description, price, main_image, is_active)
        VALUES (
            {sql_quote(payload.slug)},
            {sql_quote(payload.name)},
            {sql_quote(payload.short_description)},
            {payload.price},
            {sql_quote(payload.main_image)},
            {1 if payload.is_active else 0}
        );

        SET @fixerupper_product_id = LAST_INSERT_ID();

        INSERT INTO product_inventory (product_id, stock_quantity, location, supplier)
        VALUES (
            @fixerupper_product_id,
            {payload.stock_quantity},
            {sql_quote(payload.location)},
            {sql_quote(payload.supplier)}
        );

        INSERT INTO product_images (product_id, image_path, alt_text, sort_order)
        VALUES (@fixerupper_product_id, {sql_quote(payload.main_image)}, {sql_quote(payload.name)}, 1);

        {"".join(spec_sql)}

        COMMIT;
        SELECT @fixerupper_product_id;
        """
```

Code Screenshot 4. Product creation writes products, inventory, images and specification rows inside one START TRANSACTION / COMMIT block.

Validation, Image Handling, and Authentication

The collect_payload method is responsible for validation. It takes raw form data and builds ProductPayload. On the way it checks title length, unique slug, price format, stock as non-negative integer and other things. If something is wrong, the user sees a clear warning before sending it to the database. This is better than waiting until MySQL gives a dry error, which a normal person will read like an old writing on a stone.

With images I also had to spend some time. If the file is already inside the project folder, the application uses relative path. If the file is somewhere else on the computer, it copies it into assets/images. If the name is already taken, numeric suffix is added. This way old files are not overwritten by accident. It is a small protection, but exactly these small things make the program less fragile. Authentication uses SHA-256 hashing and hmac.compare_digest for checking password. Until login is confirmed, product management functions are blocked.

Product Workflow and Testing

When the administrator sends the form, the application first checks login status. Then it starts validation, asks for confirmation, processes the image and saves data through DatabaseClient. If everything went well, the form is reset, and the product list is refreshed. Editing works through loading existing data back into the form. Duplicate creates a new payload based on the product and generates a new slug: -copy, -copy-2 and so on. Soft delete does not kill the record completely. It makes the product inactive and puts stock to zero. This way the data does not disappear without any trace.

I did testing manually, because the application depends on a live local database. Automated tests did not enter this version. I checked program start, connection to database, admin login, loading products, saving correct and incorrect records, slug validation, external images, editing, duplicate, soft delete and list refresh. The command --check-db helped to check database connection separately from GUI. Sometimes an error appeared in one place, but the reason was sitting in a completely different place. Here I had to be a little bit like a detective, with a flashlight and patience.

Skills Developed and Evaluation. Future Improvements, Reflection, and Conclusion

If I returned to the project again, first of all I would replace mysql.exe with a normal database library. Then I would add automated tests, stronger authentication, more inventory features and interface improvements, for example image preview. This does not mean that the project is bad. It just means that now I can see better where it can go next.

The main thing I received - it was proof for myself. I was able to enter unknown territory and make a working thing. Python was standing in my list for a long time as a point “should do it one day”. In this project, “one day” finally became today. Working with a real task, not with an empty exercise, made me learn faster. Image paths, slug generation, scroll behaviour, transactions - every problem first looked like a small dragon. Then I was finding out, trying, making mistakes, and the dragon became a normal task.

I also understood better the value of structure. Separating database logic, validation logic, UI and helper functions made the code longer, but more comfortable. In the beginning, sometimes I wanted to mix everything together and finish faster. But then it became clear: a short way often leads into the deep forest. Good structure is not decoration, it is a way not to get lost.

My design experience also became useful. Even an admin tool should be understandable. A person can use it every day, and if the interface is uncomfortable, it will start to annoy very quickly. That is why colours, buttons, layout and general look were not secondary details for me.

At the beginning, I saw my main gap - not enough practice with Python. FixerUpper Inventory Desktop became a direct answer to this. It joined programming, database work, interface design and documentation. This is exactly the direction where I want to move next. It is not a perfect product. But it is real. And sometimes a real thing, with its working marks and not fully smooth corners, means more than a perfect study exercise without soul.

LINK TO DIGITAL PORTFOLIO

https://github.com/Barackudah/FixerUpper

REFERENCE LIST

Apache Friends. (n.d.). XAMPP Documentation. https://www.apachefriends.org/docs/index.html Accessed 24 May 2026.

Oracle. (2026). MySQL 9.7 Reference Manual. https://dev.mysql.com/doc/refman/9.7/en/ Accessed 27 May 2026.

The PHP Group. (2026). PHP Manual. https://www.php.net/manual/en/ Accessed 30 May 2026.

Python Software Foundation. (2026). The Python Standard Library. https://docs.python.org/3/library/index.html Accessed 2 June 2026.

Python Software Foundation. (2026). tkinter - Python interface to Tcl/Tk. https://docs.python.org/3/library/tkinter.html Accessed 5 June 2026.